Manejo de excepciones y validación de entradas (Bloque 7) / La rúbrica de calidad de código, explicada

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

La rúbrica de calidad de código, explicada

A lo largo de esta unidad ya reconociste, una por una, varias señales de qué hace que un código esté bien escrito: funciones con una sola responsabilidad, nombres que explican su propósito sin necesidad de leer el detalle, un resultado que se preserva al **refactorizar**, y un manejo de errores que informa en vez de esconder el problema.

> Esta lección junta esas señales en los cuatro criterios formales de la rúbrica que el equipo docente usa para evaluar tu módulo.

En resumen, el camino hasta aquí: en el Bloque 5 construiste una primera versión que funcionaba; en el Bloque 6 la reorganizaste en funciones separadas y confirmaste, probándola antes y después, que el resultado no había cambiado. Sin llamarlos todavía por su nombre, en esos dos bloques ya estabas practicando los mismos criterios que vas a ver formalmente a continuación.

Conocerlos con claridad ahora te permite, en la siguiente lección, programar pensando directamente en ellos, en vez de intentar ajustarse a ellos después de terminar.

📋 Los cuatro criterios, en conjunto

Ninguno de estos criterios evalúa qué tan avanzado o sofisticado es tu código. Los cuatro evalúan lo

mismo, desde ángulos distintos: qué tan fácil es para otra persona (o para ti mismo, dentro de unos meses) entender, confiar en, y seguir trabajando sobre lo que escribiste.



Responsabilidad única

Cada función hace una sola cosa, y esa cosa se puede describir en una frase corta.



Nombres descriptivos

El nombre de cada función y cada variable comunica su propósito, sin necesidad de leer su código para entender qué hace.



Comportamiento correcto y verificado

El módulo produce exactamente el resultado esperado, y ese resultado se mantuvo igual a través de los cambios que le hiciste.



Manejo adecuado de errores

Las fallas previsibles están contempladas, con mensajes claros y sin que el programa se detenga sin control.

🧩 Responsabilidad única, en más detalle

> Este criterio **evalúa si cada función hace una sola cosa, o si mezcla varias tareas que deberían estar separadas**. Ya trabajaste esto directamente en el Bloque 6, cuando tomaste tu función original del Bloque 5 y la dividiste en funciones más pequeñas, cada una con un propósito claro.

Una función que arma una URL, hace la petición, y además procesa la respuesta, todo en el mismo lugar, no cumple este criterio, incluso si el resultado final es correcto: lo que se evalúa aquí no es si funciona, es qué tan fácil es leerla y modificarla sin arriesgar el resto.

Veamos la diferencia directamente en código, comparando las dos versiones:

```
# No cumple: una sola función hace todo
 def obtener_dato_de_prueba(pregunta):
     ...  # arma la URL, envía la petición y procesa la respuesta, todo junto

 # Sí cumple: cada responsabilidad en su propia función
 def armar_peticion(pregunta): ...
 def enviar_peticion(url, body): ...
```

```
def extraer_texto(datos): ...
```

Es el mismo antes/después que ya construiste tú mismo en el Bloque 6, ahora visto directamente como criterio de evaluación.

🏷️ Nombres descriptivos, en más detalle

> Este criterio **evalúa si, con solo mirar el nombre de una función o una variable, alguien puede anticipar qué hace o qué contiene**, sin necesidad de leer una sola línea de su código. Un nombre como "procesar_respuesta_del_servicio" cumple esto; un nombre como "funcion2" o "datos" no.

Este criterio también se aplica a la consistencia: si en una parte de tu módulo llamas **"respuesta"** a lo que devuelve el servicio, y en otra parte lo llamas **"resultado"** sin razón aparente, eso agrega confusión innecesaria para quien lea tu código después.

De nuevo, la diferencia se nota apenas se pone en código:

```
# No cumple: no comunica nada por sí mismo
 def funcion2(datos):
     ...

 # Sí cumple: se entiende sin leer el código
 def procesar_respuesta_del_servicio(respuesta):
     ...
```

✅ Comportamiento correcto y verificado, en más detalle

> Este criterio **evalúa que tu módulo haga exactamente lo que se espera de él**, y **que ese comportamiento se haya confirmado**, no solo asumido. Es, en el fondo, la misma disciplina que aplicaste en el Bloque 6: probar con el mismo caso antes y después de cada cambio, para confirmar que el resultado sigue siendo idéntico.

Un módulo que "probablemente funciona", porque nunca se volvió a probar después de los últimos cambios, no cumple este criterio con la misma fuerza que uno donde ese comportamiento fue efectivamente verificado.



🛡️ Manejo adecuado de errores, en más detalle

> Este último criterio es el que vas a aplicar de forma más directa en la última lección de esta unidad. **Evalúa si tu módulo anticipa las fallas más probables** (un servicio que no responde, una respuesta en un formato inesperado), y si responde a ellas de una forma que informa con claridad, en vez de detenerse sin control o de esconder el problema en silencio.

Este criterio no pide que tu módulo prevea absolutamente todo lo que podría salir mal, algo que en la práctica es imposible. Pide que las fallas más razonables de anticipar (las mismas que ya viste en esta unidad: error de conexión, error de formato, dato faltante) estén efectivamente contempladas.

```python
# No cumple: se detiene sin control
resultado = obtener_dato_de_prueba(pregunta)
print(resultado)

# Sí cumple: informa el problema en vez de detenerse
try:
    resultado = obtener_dato_de_prueba(pregunta)
    print(resultado)
except requests.exceptions.ConnectionError:
    print("No se pudo conectar con el servicio. Intenta de nuevo en unos segundos.")
```

📋 Cómo se usa esta rúbrica al revisar tu módulo

El equipo docente no revisa estos cuatro criterios como una lista de aprobado o reprobado línea por línea, sino como una mirada conjunta sobre la calidad de tu **módulo**.

> ✨ Un **módulo** que cumple bien los cuatro criterios se nota, casi de inmediato, apenas se empieza a leer: se entiende rápido, inspira confianza, y es fácil imaginar cómo seguir construyendo sobre él.



Antes de seguir a la última lección, revisa tu propio módulo contra estos cuatro criterios:

> **Responsabilidad única**: ¿cada función de tu módulo hace una sola cosa, y la puedes describir en una frase corta?

> **Nombres descriptivos**: ¿se entiende qué hace cada función y qué contiene cada variable, sin necesidad de leer su código?

> **Comportamiento correcto y verificado**: ¿probaste tu módulo con el mismo caso después de tus últimos cambios, y el resultado sigue siendo el esperado?

> **Manejo adecuado de errores**: ¿tu módulo informa con claridad cuando algo falla, en vez de detenerse sin control o esconder el problema?

Siguiente lección

Excepciones, validación y commit · Actividad Core

Con estos cuatro criterios ya claros, la última lección de esta unidad es donde los aplicas directamente sobre tu propio módulo, incorporando manejo de excepciones y validación, y cerrando formalmente el entregable de la **Unidad U1.3**.

Última modificación: jueves, 24 de septiembre de 2026, 15:45